

Senior PHP Developer — Take-Home Assignment

Before you start

You may use AI tools. Claude, ChatGPT, Copilot — all fine. We use them too.

This assignment is not just about the code. It is built to look like a normal day in this job: one real feature to ship, plus two unrelated interrupts that show up while you're mid-flow, because that is what actually happens here. We are watching how you prioritize, how fast you move with AI without losing quality, and how clearly you report status in English — because you will be doing all three, most days, for people who are not in the room with you.

You will be asked to defend every decision in a follow-up call.

Time budget: 3–4 hours total. This is a ceiling, not a target. If you triage well, you will not need all of it for full marks — a well-reasoned partial submission beats a rushed complete one.

The setup

You're an embedded contractor covering three small client projects this week. All three tickets below are open on your board **right now**. Nobody is going to tell you which order to do them in — that judgment call is part of what's being graded.

Ticket A — Order Sync Webhook (the main task, ~65% of your time)

Client: "Fenwick Retail"

Build a webhook receiver that syncs paid orders from a payment provider into an internal **orders** table.

POST /webhooks/payment-provider

A Postgres/MySQL schema (your choice) and a sample payload are up to you to define — the provider's real webhook format is not attached, because it never is on day one of a real ticket. Model it on any major payment provider's webhook shape you're familiar with (Stripe-style is fine) and state the assumption.

Requirements

1. Receiving a **payment.succeeded** event creates an order record if one doesn't already exist for that payment.
2. The provider is known to **retry webhook delivery** on any non-2xx response or timeout, including after it already succeeded on our end.
3. Must verify the webhook came from the provider, not just anyone who can find the URL.
4. Failed processing should be visible somewhere a human will actually check it.

That's the whole spec. It's incomplete on purpose.

Ticket B — Interrupt (arrives ~20–30 minutes into your session)

Client: "Northgate Logistics" — flagged URGENT by their PM

Their `shipment_date` field is silently wrong for a subset of records. Northgate's legacy import script parses dates with a hardcoded format assumption, and a batch of records from a different regional office uses a different date format. Some dates have already been silently misparsed and written to the database — not just rejected, **wrong**.

Fix the parser, and decide for yourself whether "fix the parser" is actually enough given what you just read. Northgate needs an answer today, not a lecture — say what you're doing and why, briefly.

Ticket C — Interrupt (arrives ~2 hours in)

Client: "Marlow Events" — small, "whenever you get a sec"

They need a read-only endpoint:

GET /api/events/upcoming

Returns their next 10 events. No auth system exists for this client yet. Marlow's dev is not available today to clarify.

What to deliver?

1. Working code

For all three tickets. Whatever tests you think are warranted — but "warranted" should mean something different for a payment webhook than for a read-only events list, and we're watching whether you treat them differently.

2. PROGRESS.md

A running, timestamped log (real or simulated timestamps are both fine) of how you moved between the three tickets — what you were doing when each interrupt landed, what you paused, what you finished first and why. This is not a diary; a few lines per switch is enough. We want to see the triage, not a transcript.

3. DECISIONS.md

Answer these:

1. **What did each ticket's spec not tell you, and what did you assume?** List the assumption you're least confident about, across all three.

2. **What did you use AI for on Ticket A specifically, and where did you override it?** Be concrete. What did it generate that was wrong, incomplete, or would have shipped a bug if you hadn't caught it?

3. What would break first if Fenwick's order volume grew 100x, and how would you know before a customer told you?

4. What did you deliberately not build, across all three tickets, and why?

4. **STATUS_REPORT.md** — written in clear English, for a non-technical client contact

Three short paragraphs, one per client, as if you were sending an end-of-day update to each client's business contact (not their developer). State what's done, what's outstanding, and anything they need to decide or provide. No jargon they wouldn't understand. This is being read for clarity as much as for content.

How is this graded?

Signal	Strong	Weak
Prioritization	Reasoned order, stated trade-offs, nothing silently dropped	Everything attempted shallowly, or tickets done in arrival order with no judgment shown
AI usage	Names exactly what it got wrong on the hard ticket (A)	"Used it for boilerplate," or no visible override anywhere
Ticket A correctness	Idempotency and signature verification both actually handled, not just mentioned	Retried webhook would double-process; anyone can POST to the endpoint
Ticket B judgment	Recognizes and flags the corrupted historical data, not just the parser bug	Fixes the parser going forward, says nothing about data already wrong
Ticket C judgment	Flags the missing auth explicitly rather than silently trusting the request	Ships an open endpoint with no comment, or invents auth nobody asked for without saying so
English communication	STATUS_REPORT reads naturally, no technical jargon, a business owner would understand it in one pass	Copy-pasted technical language, unclear, or grammatically hard to follow
Testing	Tests match the actual risk of each ticket — heaviest on the payment webhook	Same shallow test (or none) on all three regardless of stakes

A candidate who ships all three competently with a clear report will outrank one who ships a beautiful Ticket A and drops B and C silently. Dropping something and saying so is a pass. Dropping something and not mentioning it is not.

Submission

Reply with a repository link (or a zip). Include setup instructions — if we can't run it in five minutes, we grade it as it arrives.

The follow-up call

Twenty-five minutes, screen shared. Expect:

- Walk me through what was happening in your head when Ticket B landed.
- Show me the exact place the AI got the webhook idempotency wrong.
- If Marlow calls tomorrow asking for auth, what do you build and how long does it take?
- Read me your STATUS_REPORT for Fenwick like you're on the phone with their owner right now.

Bring **PROGRESS.md** and **DECISIONS.md**. They're the agenda.

Questions about any ticket are welcome — asking a good one, especially about Ticket C's missing auth, is itself a positive signal.